



Inside a Kubernetes Cluster

Week 1

Index Table

1	What Is a Kubernetes Cluster?
2	Control Plane vs Worker Nodes
3	Control Plane Components (API Server, Scheduler, Controllers, etcd)
4	Worker Node Components (kubelet, container runtime, kube-proxy)
5	What Kubernetes Automates for You
6	What Kubernetes Does Not Do
7	A Real-World Cluster Scenario
8	Key Takeaways & What's Next

Index 1

What Is a Kubernetes Cluster?

Understanding how Kubernetes groups, runs, and manages applications

A Kubernetes Cluster is:

A group of machines working together to run containerized applications reliably.

- Runs containers at scale
- Manages failures automatically
- Keeps applications available

A cluster is made of two main parts:

Control Plane

- The brain decides what should happen.

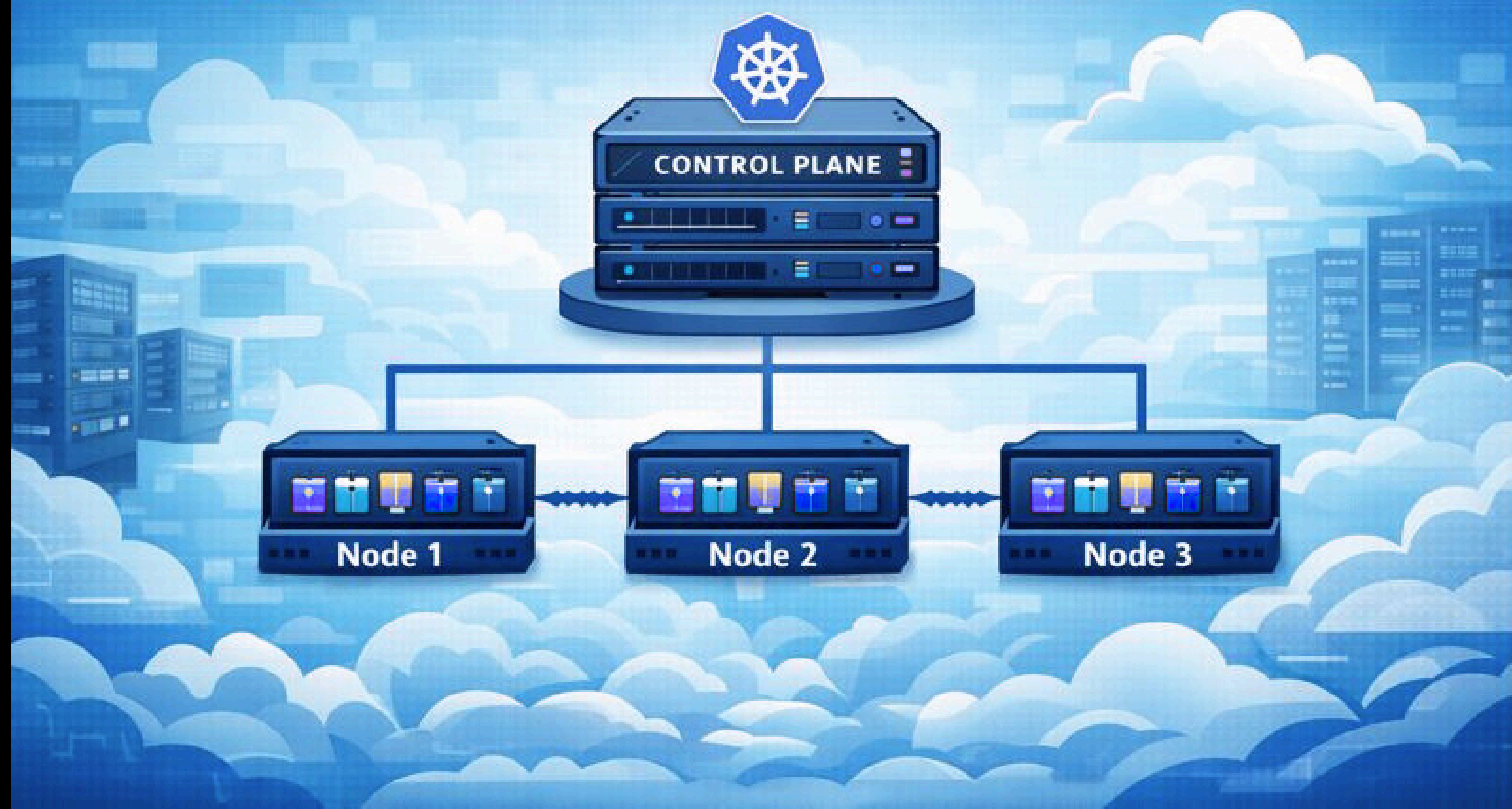
Worker Nodes

- The muscle (does the actual work)

Kubernetes separates decision-making from execution.



Kubernetes Cluster



What a Cluster Is Not

A Kubernetes cluster is not:

- Just Docker
- Just servers
- A single machine
- A replacement for good app design

Kubernetes manages infrastructure.

Index 2

Control Plane vs Worker Nodes

Who decides and who does the work in Kubernetes

The Big Split



Control Plane

Decides what should happen

Plans, schedules, and monitors

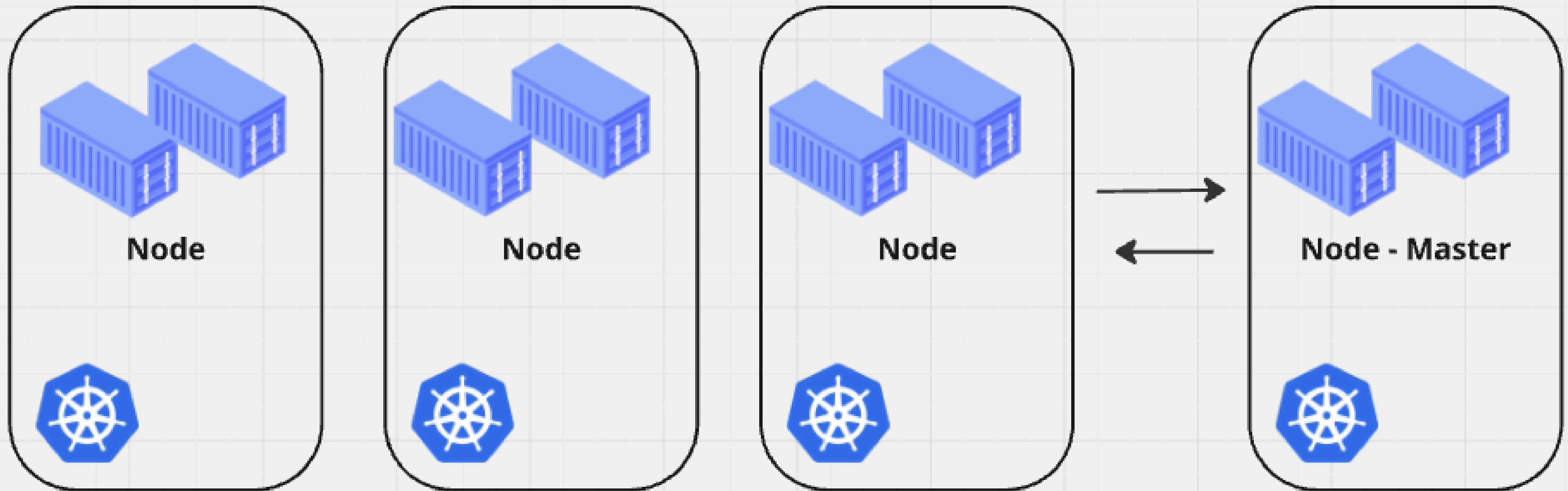
Worker Nodes

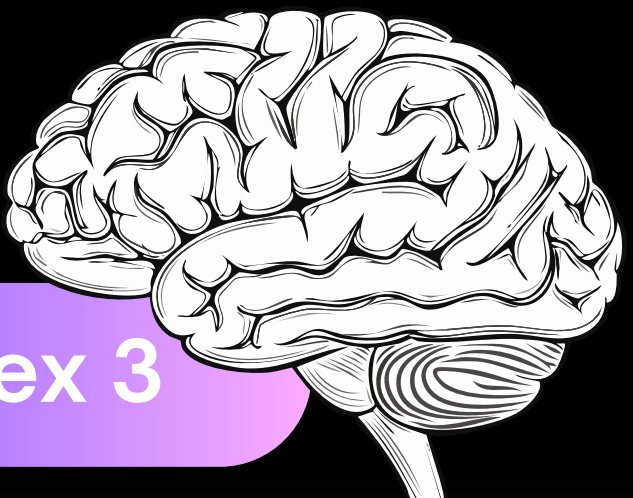
Execute what actually happens

Runs Pods and containers

Kubernetes separates decision-making from execution.

Kubernetes Cluster - Master

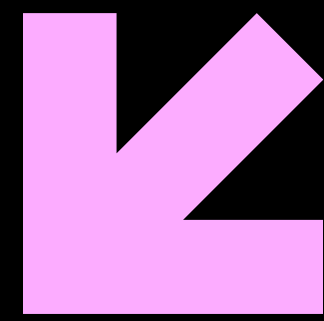




Index 3

Control Plane Components

The brain of Kubernetes that manages cluster state and scheduling



Control Plane Components

API Server

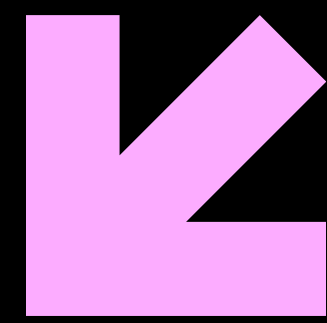
The central entry point of the cluster.

All requests from users, tools, and components go through the API Server, where they are validated and processed before any action happens.

Scheduler

Chooses the best worker node for each Pod.

It evaluates resources, constraints, and policies, then assigns Pods to nodes, but does not run them itself.



Control Plane Components

Controller Manager

Keeps the cluster in the desired state.

It continuously monitors resources and takes action to address discrepancies between what you want and what actually exists.

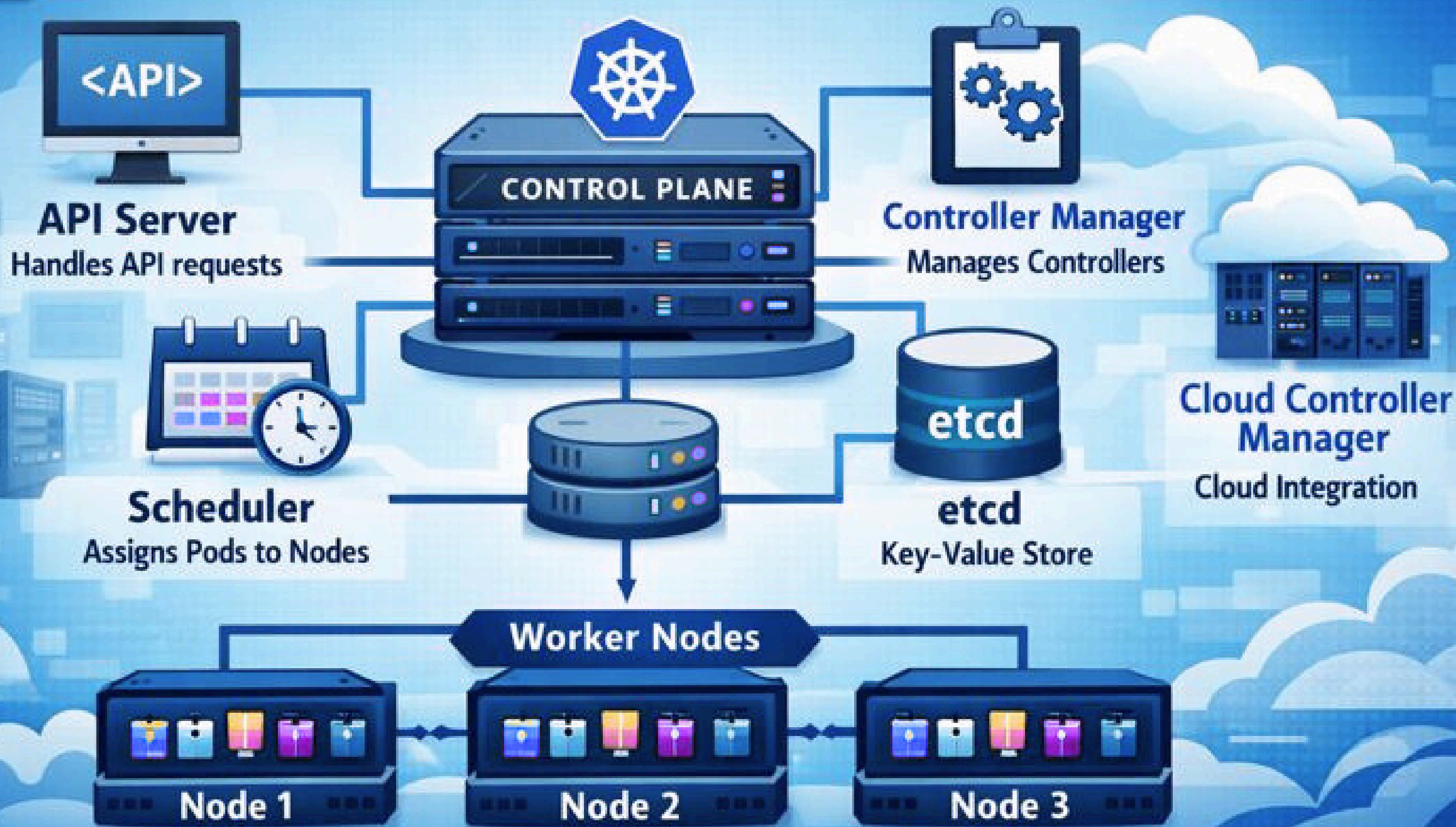
etcd

The cluster's source of truth. A distributed key-value store that holds all cluster state and configuration data.

The Control Plane decides what should happen, not how it runs.



Kubernetes Control Plane



Index 4

Worker Node Components

Where your applications actually run inside the cluster

Worker Node Components

kubelet

The node's main agent.
It watches the API Server and
ensures Pods are running exactly
as defined.

Container Runtime

Runs the containers.
Pulls images, starts containers,
and manages their lifecycle (e.g.,
containerd, CRI-O).

Worker Node Components

kube-proxy

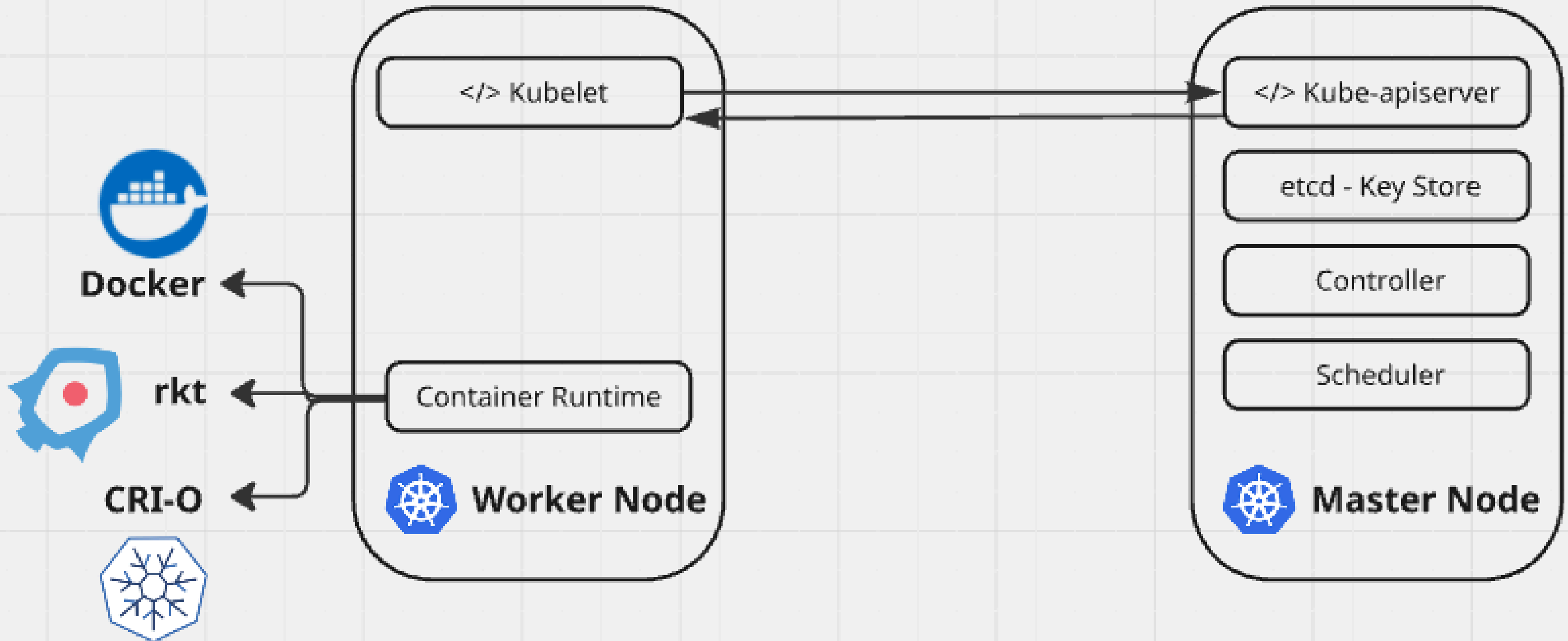
Handles network traffic.
Manages routing and load-balancing so Services can reach Pods reliably.

Pods

Where applications live.
Each Pod groups one or more containers that share networking and storage.

Worker nodes don't decide, they execute.

Master vs Worker Node

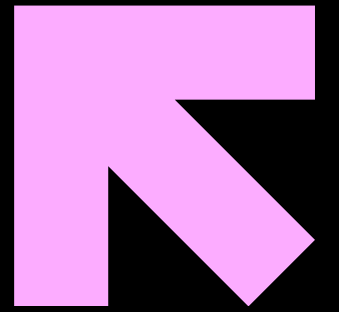


Index 5

What Kubernetes Automates for You

The operational tasks Kubernetes handles so you don't have to

Kubernetes Automatically Handles



Scheduling

Places pods on the right nodes based on resources & constraints.

Self-healing

Restarts failed containers and replaces unhealthy pods.

Scaling

Adjusts the number of running pods based on demand.

Rollouts & Rollbacks

Updates applications gradually and safely.

Service Discovery & Load Balancing

Keeps services reachable even as pods change.

👉 You describe the desired state, Kubernetes keeps it true.

Index 6

What Kubernetes Does Not Do

Responsibilities that remain outside Kubernetes' scope

Kubernetes Does Not

- Write or fix your application code
- Design your application architecture
- Decide how your app should behave logically
- Automatically make bad apps scalable
- Replace good engineering practices

You are still responsible for:

- Application security (code, auth logic, vulnerabilities)
- Performance tuning & resource planning
- Data design, backups, and migrations
- Testing, CI/CD pipelines, and release strategy
- Observability decisions (what to monitor & why)

Index 7

A Real-World Cluster Scenario

Connecting Kubernetes concepts to a real application example

Lab Worksheet

Index 8

Key Takeaways & What's Next

Core learnings and upcoming Kubernetes topics

Key Takeaways

- Kubernetes is not just containers; it's a control system
- You describe the desired state, and Kubernetes keeps it true
- Control Plane decides, Worker Nodes execute
- Failures are expected; self-healing is built in
- Scaling and reliability are handled by configuration, not servers

- Next Session: Kubernetes Networking
- Pods, Services, Ingress, and traffic flow
- How applications talk to each other
- How traffic enters and moves inside the cluster
- When networking design matters for production

What's Next

Thank You